EmbeddingBag#

https://pytorch.org/docs/stable/generated/torch.nn.EmbeddingBag.html#torch.r

Description:

Compute sums or means of ‘bags’ of embeddings, without instantiating the intermediate embeddings. For bags of constant length, no per_sample_weights, no indices equal to padding_idx, and with 2D inputs, this class with mode="sum" is equivalent to Embedding followed by torch.sum(dim=1), with mode="mean" is equivalent to Embedding followed by torch.mean(dim=1), with mode="max" is equivalent to Embedding followed by torch.max(dim=1).

Function Signature

```
class torch.nn.EmbeddingBag(num_embeddings, embedding_dim,
max_norm=None, norm_type=2.0, scale_grad_by_freq=False,
mode='mean', sparse=False, _weight=None,
include_last_offset=False, padding_idx=None, device=None,
dtype=None)[source]#

Parameters

Variables

forward(input, offsets=None, per_sample_weights=None)
[source]#

Parameters
```

Returns:

EmbeddingBag

Code Examples

```
>>> # an EmbeddingBag module containing 10 tensors of size 3
>>> embedding_sum = nn.EmbeddingBag(10, 3, mode='sum')
>>> # a batch of 2 samples of 4 indices each
>>> input = torch.tensor([1, 2, 4, 5, 4, 3, 2, 9],
dtype=torch.long)
>>> offsets = torch.tensor([0, 4], dtype=torch.long)
>>> embedding_sum(input, offsets)
tensor([[-0.8861, -5.4350, -0.0523],
        [ 1.1306, -2.5798, -1.0044]])

>>> # Example with padding_idx
>>> embedding_sum = nn.EmbeddingBag(10, 3, mode='sum',
padding_idx=2)
>>> input = torch.tensor([2, 2, 2, 2, 4, 3, 2, 9],
dtype=torch.long)
>>> offsets = torch.tensor([0, 4], dtype=torch.long)
>>> embedding_sum(input, offsets)
tensor([[ 0.0000,  0.0000,  0.0000],
        [-0.7082,  3.2145, -2.6251]])

>>> # An EmbeddingBag can be loaded from an Embedding like so
>>> embedding = nn.Embedding(10, 3, padding_idx=2)
>>> embedding_sum = nn.EmbeddingBag.from_pretrained(
        embedding.weight,
        padding_idx=embedding.padding_idx,
        mode='sum')
```

```
>>> # FloatTensor containing pretrained weights
>>> weight = torch.FloatTensor([[1, 2.3, 3], [4, 5.1, 6.3]])
>>> embeddingbag = nn.EmbeddingBag.from_pretrained(weight)
>>> # Get embeddings for index 1
>>> input = torch.LongTensor([[1, 0]])
>>> embeddingbag(input)
tensor([[ 2.5000,  3.7000,  4.6500]])
```

Notes & Warnings

NOTE

Note A few notes about input and offsets: input and offsets have to be of the same type, either int or long If input is 2D of shape (B, N), it will be treated as B bags (sequences) each of fixed length N, and this will return B values aggregated in a way depending on the mode. offsets is ignored and required to be None in this case. If input is 1D of shape (N), it will be treated as a concatenation of multiple bags (sequences). offsets is required to be a 1D tensor containing the starting index positions of each bag in input. Therefore, for offsets of shape (B), input will be viewed as having B bags. Empty bags (i.e., having 0-length) will have returned vectors filled by zeros.

Detailed Documentation

Documentation

Compute sums or means of ‘bags’ of embeddings, without instantiating the intermediate embeddings.

For bags of constant length, no per_sample_weights, no indices equal to padding_idx, and with 2D inputs, this class

with mode="sum" is equivalent to Embedding followed by torch.sum(dim=1),

with mode="mean" is equivalent to Embedding followed by torch.mean(dim=1),

with mode="max" is equivalent to Embedding followed by torch.max(dim=1).

However, EmbeddingBag is much more time and memory efficient than using a chain of these operations.

EmbeddingBag also supports per-sample weights as an argument to the forward pass. This scales the output of the Embedding before performing a weighted reduction as specified by mode. If per_sample_weights is passed, the only supported mode is "sum", which computes a weighted sum according to per_sample_weights.

num_embeddings (int) – size of the dictionary of embeddings

embedding_dim (int) – the size of each embedding vector

max_norm (float, optional) – If given, each embedding vector with norm larger than max_norm is renormalized to have norm max_norm.

norm_type (float, optional) – The p of the p-norm to compute for the max_norm option. Default 2.

scale_grad_by_freq (bool, optional) – if given, this will scale gradients by the inverse of frequency of the words in the mini-batch. Default False. Note: this option is not supported when mode="max".

mode (str, optional) – "sum", "mean" or "max". Specifies the way to reduce the bag. "sum" computes the weighted sum, taking per_sample_weights into consideration. "mean" computes the average of the values in the bag, "max" computes the max value over each bag. Default: "mean"

sparse (bool, optional) – if True, gradient w.r.t. weight matrix will be a sparse tensor. See Notes for more details regarding sparse gradients. Note: this option is not supported when mode="max".

include_last_offset (bool, optional) – if True, offsets has one additional element, where the last element is equivalent to the size of indices. This matches the CSR format.

padding_idx (int, optional) – If specified, the entries at padding_idx do not contribute to the gradient; therefore, the embedding vector at padding_idx is not updated during training, i.e. it remains as a fixed “pad”. For a newly constructed EmbeddingBag, the embedding vector at padding_idx will default to all zeros, but can be updated to another value to be used as the padding vector. Note that the embedding vector at padding_idx is excluded from the reduction.

weight (Tensor) – the learnable weights of the module of shape (num_embeddings, embedding_dim) initialized from N(0,1)\mathcal{N}(0, 1)N(0,1).

Forward pass of EmbeddingBag.

input (Tensor) – Tensor containing bags of indices into the embedding matrix.

offsets (Tensor, optional) – Only used when input is 1D. offsets determines the starting index position of each bag (sequence) in input.

per_sample_weights (Tensor, optional) – a tensor of float / double weights, or None to indicate all weights should be taken to be 1. If specified, per_sample_weights must have exactly the same shape as input and is treated as having the same offsets, if

those are not None. Only supported for mode='sum'.

Tensor output shape of (B, embedding_dim).

A few notes about input and offsets:

input and offsets have to be of the same type, either int or long

If input is 2D of shape (B, N), it will be treated as B bags (sequences) each of fixed length N, and this will return B values aggregated in a way depending on the mode. offsets is ignored and required to be None in this case.

If input is 1D of shape (N), it will be treated as a concatenation of multiple bags (sequences). offsets is required to be a 1D tensor containing the starting index positions of each bag in input. Therefore, for offsets of shape (B), input will be viewed as having B bags. Empty bags (i.e., having 0-length) will have returned vectors filled by zeros.

Create EmbeddingBag instance from given 2-dimensional FloatTensor.

embeddings (Tensor) – FloatTensor containing weights for the EmbeddingBag. First dimension is being passed to EmbeddingBag as ‘num_embeddings’, second as ‘embedding_dim’.

freeze (bool, optional) – If True, the tensor does not get updated in the learning process. Equivalent to embeddingbag.weight.requires_grad = False. Default: True

max_norm (float, optional) – See module initialization documentation. Default: None

norm_type (float, optional) – See module initialization documentation. Default 2.

scale_grad_by_freq (bool, optional) – See module initialization documentation. Default False.

mode (str, optional) – See module initialization documentation. Default: "mean"

sparse (bool, optional) – See module initialization documentation. Default: False.

include_last_offset (bool, optional) – See module initialization documentation. Default: False.

padding_idx (int, optional) – See module initialization documentation. Default: None.
